

# STELLAR CODE

Security Division

Smart Contract Security Audit Report

## HNTRMembership Protocol

Tiered Membership & 12-Level Unilevel Commission Distribution

|                    |                                                                       |
|--------------------|-----------------------------------------------------------------------|
| Client             | HNTR                                                                  |
| Repository         | hntr-web3                                                             |
| Contracts in Scope | HNTRMembership.sol, IHNTRMembership.sol                               |
| Commit / Snapshot  | Working tree as provided, 18 July 2026                                |
| Target Chain       | Ethereum mainnet (USDT / USDC, 6 decimals)                            |
| Toolchain          | Solidity ^0.8.24, Foundry, OpenZeppelin Contracts v5                  |
| Audited by         | Stellar Code Security Division                                        |
| Review Type        | Manual line-by-line review, threat modelling, accounting verification |
| Date of Issue      | 18 July 2026                                                          |
| Version            | 2.0 — Security-Focused Edition                                        |
| Classification     | Confidential — Client Distribution Only                               |

5 SECURITY FINDINGS · 0 CRITICAL · 4 HIGH · 1 MEDIUM

# Table of Contents

---

- 1. Document Detail & Report History
- 2. Executive Summary
- 3. Scope
- 4. Severity Classification
- 5. Protocol Overview
  - 5.1 Membership & Payment Flow
  - 5.2 Commission Distribution Model
  - 5.3 Fund Accounting Verification
- 6. Security Findings
  - SEC-01 Signing key compromise redirects commission stream — High
  - SEC-02 Ownership permanently bound to deploying EOA — High
  - SEC-03 No pause and no rescue — High
  - SEC-04 Unmeasured token receipt — insolvency risk — High
  - SEC-05 No duplicate or self upline rejection — Medium
- 7. Verified Secure — Attack Surface Cleared
- 8. Trust Model
- 9. Remediation Roadmap
- 10. Conclusion
- 11. Disclaimer
- A. Appendix — Non-Security Observations

# 1. Document Detail & Report History

|                   |                                              |
|-------------------|----------------------------------------------|
| Document Title    | HNTRMembership Smart Contract Security Audit |
| Document Owner    | Stellar Code Security Division               |
| Prepared For      | HNTR Protocol Team                           |
| Assessment Window | July 2026                                    |
| Report Status     | Final — v2.0                                 |
| Distribution      | Client engineering and leadership only       |

| Version | Date        | Change Description                                                                                                                                                                                                                                                      |
|---------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0     | 18 Jul 2026 | Initial issue — 25 findings across all categories                                                                                                                                                                                                                       |
| 1.1     | 18 Jul 2026 | Decimal-assumption finding downgraded following client confirmation of Ethereum mainnet as the deployment target                                                                                                                                                        |
| 1.2     | 18 Jul 2026 | Locked-commission finding downgraded following client confirmation that the 20% is settled off-chain by design                                                                                                                                                          |
| 2.0     | 18 Jul 2026 | Restructured at client request to cover security-relevant findings only. Five findings retained. Items with no path to loss of funds or loss of control were reclassified as code-quality observations and moved to Appendix A. No finding was removed from the record. |

*A note on this edition.* Version 1.0 reported 25 findings. On review, the majority described code quality, gas efficiency and maintainability rather than security — legitimate engineering feedback, but presenting it alongside genuine attack paths obscured the four issues that actually matter. This edition covers only findings with a demonstrable path to loss of funds or loss of protocol control. Everything else is preserved in Appendix A.

## 2. Executive Summary

Stellar Code assessed **HNTRMembership**, a tiered membership contract that accepts USDT or USDC on Ethereum mainnet and distributes commissions across a twelve-level unilevel structure with dynamic compression.

**No critical vulnerability was found, and the core accounting is sound.** We verified the commission arithmetic symbolically and confirmed it balanced: retained funds always equal recorded liabilities, and the level percentages sum to exactly 65% (Section 5.3). SafeERC20 is used throughout, so USDT's non-standard return value is handled correctly. Checks-effects-interactions is observed in the withdrawal path. The signature payload binds chain ID, contract address and an operation tag, closing cross-chain, cross-contract and cross-function replay. Critically, **there is no function — privileged or otherwise — that can move a member's accrued commissions to anyone but that member.** Neither the owner nor the company wallet can drain the contract. Section 7 details the attack surface we cleared.

**The findings that remain are about recoverability, not exploitability.** There is no way for an unprivileged attacker to steal funds or obtain a free membership. What concerns us is that the protocol has no mechanism to recover from anything going wrong. A single off-chain key authorises every commission payload, with no nonce and no way to revoke signatures already issued (SEC-01). Ownership is permanently bound to the deploying EOA, so the role can never be moved to a multisig and a compromised signer can never be rotated if that key is lost (SEC-02). Nothing can be paused and nothing can be rescued (SEC-03).

Those three compound. Individually each is a common finding with a standard fix. Together they mean that on the day something goes wrong — a leaked signing key, a frozen treasury address, a flaw discovered post-launch — the only available response is public abandonment of the contract, stranding every accrued balance. SEC-04 describes the most plausible such day: the distribution logic assumes the tokens received equal the amount requested, which is true today and stops being true the moment Tether enables its dormant transfer fee.

### PRIMARY RECOMMENDATION: ADD Ownable2Step AND Pausable BEFORE DEPLOYMENT

SEC-02 and SEC-03 are the cheapest findings in this report to fix and the only two that **cannot be fixed after deployment**. Two inherited contracts and a handful of modifiers convert an irreversible protocol into a recoverable one. We would not deploy without them.

| ID     | Finding                                                                            | Severity |
|--------|------------------------------------------------------------------------------------|----------|
| SEC-01 | Compromise of the signing key redirects the entire commission stream               | High     |
| SEC-02 | Ownership is permanently bound to the deploying EOA and can never reach a multisig | High     |
| SEC-03 | No pause and no rescue — the protocol has no incident response capability          | High     |
| SEC-04 | Distribution assumes tokens received equals amount requested — insolvency risk     | High     |
| SEC-05 | No duplicate or self-referential upline rejection                                  | Medium   |

### 3. Scope

| File                              | SLOC | Role                               | In Scope              |
|-----------------------------------|------|------------------------------------|-----------------------|
| src/HNTRMembership.sol            | 336  | Core membership & commission logic | Yes — primary         |
| src/IHNTRMembership.sol           | 53   | Interface, enums, User struct      | Yes — primary         |
| script/DeployHNTRMembership.s.sol | 91   | Foundry deployment & configuration | Supporting            |
| test/Mocks.sol                    | 44   | Test doubles                       | Supporting            |
| test/HNTR.t.sol                   | —    | Foundry test suite                 | Supporting            |
| OpenZeppelin Contracts v5         | —    | SafeERC20, ECDSA, MessageHashUtils | Out — assumed correct |

**Material out-of-scope dependency.** The off-chain backend that produces commission signatures was not assessed. Because the contract delegates all commission authorisation to that component without independent validation, its security is not a peripheral concern — it *is* the security of the commission system. SEC-01 describes the on-chain consequences of its compromise, but the probability of that compromise is determined entirely by code we did not review. We recommend a separate assessment of the signing service and its key management.

Also out of scope: the front-end application, deployment infrastructure, and the economic sustainability of the commission model.

### 4. Severity Classification

| Severity | Definition                                                                                                                                                               |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Critical | Direct, permanent loss of funds or complete protocol compromise, exploitable by an unprivileged actor.                                                                   |
| High     | Significant loss of funds, protocol halt, or irreversible loss of control — possibly requiring privileged access, a specific configuration, or an external precondition. |
| Medium   | Limited or conditional fund impact, or a missing invariant that materially increases the blast radius of another finding.                                                |
| Excluded | Code quality, gas efficiency, documentation and maintainability items with no path to loss of funds or control. See Appendix A.                                          |

## 5. Protocol Overview

### 5.1 Membership & Payment Flow

A member calls `purchaseMembership` with a tier, payment token, upline list, matching rank list, deadline and a signature from `companyWallet`. The contract verifies the signature, records the membership, pulls the tier price and distributes it. `upgradeMembership` follows the same path but charges only the tier price difference. Both require `msg.sender == user`, so the buyer always pays for themselves — the member covers gas, the backend supplies authorisation.

| Tier     | Price (USDC/USDT) | Gates Commission Levels                        |
|----------|-------------------|------------------------------------------------|
| BRONZE   | 50                | 4                                              |
| SILVER   | 250               | 5–6                                            |
| GOLD     | 750               | 7–10                                           |
| PLATINUM | 1,500             | 11–12                                          |
| DIAMOND  | 2,500             | — (qualifies everywhere via $\geq$ comparison) |

### 5.2 Commission Distribution Model

Each payment splits into four fixed allocations and one variable commission stream. The commission loop awards level  $n$  to the first upline satisfying both the tier and rank gate for that level — **dynamic compression**: unqualified uplines are skipped without consuming a level.

| Allocation           | Share    | Destination                                          |
|----------------------|----------|------------------------------------------------------|
| Treasury             | 25%      | treasuryWallet (5% base + 20% reallocated from pool) |
| Leadership Pool      | 5%       | leadershipWallet                                     |
| Achievement Bonus    | 5%       | achievementWallet                                    |
| Unilevel Commissions | 65%      | 80% retained as withdrawable / 20% to poolWallet     |
| Breakage             | residual | treasuryWallet — undistributed portion of the 65%    |

  

| Level    | 1  | 2  | 3 | 4   | 5   | 6   | 7   | 8   | 9   | 10  | 11  | 12  |
|----------|----|----|---|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Share %  | 15 | 15 | 8 | 5   | 4   | 4   | 4   | 2   | 2   | 2   | 2   | 2   |
| Min Tier | —  | —  | — | BRZ | SLV | SLV | GLD | GLD | GLD | GLD | PLT | PLT |
| Min Rank | —  | —  | — | SCT | TRK | TRK | RNG | RNG | RNG | RNG | HTR | HTR |

### 5.3 Fund Accounting Verification

Whether the commission pool stays solvent is the single most important property of this contract, so we verified it explicitly rather than assuming it. For a payment of amount  $A$  with total commission distributed  $D$ :

```
Outflow = 0.25A      (treasury)
        + 0.05A      (leadership)
        + 0.05A      (achievement)
        + 0.20D      (locked -> poolWallet)
        + (0.65A - D) (breakage -> treasury)
        = A - 0.80D

Retained = A - Outflow = 0.80D
Recorded liability (withdrawableCommissions) = 0.80D

Retained == Liability    LEDGER IS SOLVENT
Sum of levelPercentages = 15+15+8+5+4+4+4+2+2+2+2 = 65    CORRECT
```

The accounting holds. Because every split floors independently, total outflow is always less than or equal to  $A$ , so rounding can never cause over-distribution — the residue accumulates in the contract's favour. **This proof depends on one assumption: that the contract receives exactly what it requested.** SEC-04 is the finding that breaks that assumption, and it is the only realistic route to insolvency we identified.

## 6. Security Findings

### SEC-01 Compromise of the signing key redirects the entire commission stream

**HIGH**

|            |                                            |
|------------|--------------------------------------------|
| Location   | HNTRMembership.sol L173–208, L106–109      |
| Category   | Signature Verification / Key Compromise    |
| Impact     | High — loss of all future commission value |
| Likelihood | Medium — depends on backend key hygiene    |

#### Description

Every purchase and upgrade requires an ECDSA signature from `companyWallet` over the full commission payload, including the upline list and each upline's rank. The contract performs no independent validation of that payload — it does not check that the uplines bear any relationship to the buyer, that they are distinct, or that the ranks are genuine. The signer alone decides who gets paid.

An attacker holding that key signs a payload naming twelve addresses they control, at rank HUNTER, for every incoming sale. They collect 65% of all protocol revenue from that moment on. No member action, owner action, or contract state can stop it: there is no pause (SEC-03), and rotating `companyWallet` requires the owner key, which is a single permanent EOA (SEC-02).

The signed struct commits to `chainid`, `address(this)` and an operation tag, which correctly closes cross-chain, cross-contract and cross-function replay. **It contains no nonce.** Same-message replay happens to be blocked today by the `tier == Tier.NONE` and `newTier > u.tier` state guards — but that is incidental, not designed. Any future change to those guards reopens replay silently. More immediately, there is no way to invalidate a signature that has already been issued: if a batch of signed payloads leaks, every one of them stays valid until its deadline expires, and rotating the signer does not revoke them.

#### Proof of Concept

```
bytes32 structHash = keccak256(abi.encode(
    user, tier, uplines, ranks, token,
    deadline, block.chainid, address(this), operation
));
// <-- no nonce

address signer = ECDSA.recover(digest, signature);
require(signer == companyWallet, "Invalid signature");
// ..no check that uplines relate to user, are distinct,
// or that the claimed ranks were ever earned

// With the key: sign(user=victim,
//                  uplines=[attacker x12],
//                  ranks=[HUNTER x12])
// -> attacker receives 65% of every subsequent sale
```

#### Recommendation

1. Add a **per-user nonce** to the signed struct and increment it on use, so each signature is single-use by construction rather than by side effect.
2. Add a **global epoch counter** included in the hash, which the owner can bump in one transaction to invalidate every outstanding signature at once. This is the control that turns a key leak from unrecoverable into a bad afternoon.
3. **Keep deadlines short** — minutes, not days. This bounds the damage window for any leaked but unsubmitted payload.
4. **Support multiple signers** via `mapping(address => bool) isSigner` with individual revocation, rather than a single address, and hold the keys in an HSM or KMS.



**5. Migrate to EIP-712.** The current EIP-191 `personal_sign` encoding is cryptographically sound, but the operator signs an opaque digest — no human or hardware wallet can see what is being authorised. Typed data makes the payload reviewable at the point of signing.

**SEC-02 Ownership is permanently bound to the deploying EOA and can never reach a multisig****HIGH**

|            |                                           |
|------------|-------------------------------------------|
| Location   | HNTRMembership.sol L24, L72–75, L88       |
| Category   | Access Control / Key Management           |
| Impact     | High — permanent loss of protocol control |
| Likelihood | Medium                                    |

**Description**

owner is assigned in the constructor and there is no `transferOwnership`, no two-step handover, and no `renounce`. The privileged role is bound forever to the hot key that ran the deploy script.

This is a security issue rather than a convenience issue for two reasons. First, the standard mitigation for single-key risk — move the role to a Safe multisig shortly after deployment — is unavailable. The protocol is permanently secured by whatever operational hygiene surrounds one EOA. Second, that key is the *only* way to rotate `companyWallet`. If it is lost, a compromised signer (SEC-01) can never be replaced and the protocol must be abandoned, stranding every accrued commission balance in the contract.

**Proof of Concept**

```
address public owner;

constructor(address _usdt, address _usdc) {
    owner = msg.sender;    // set once, forever
}

modifier onlyOwner() { require(msg.sender == owner, "Not owner"); _; }

// no transferOwnership()
// no acceptOwnership()
// no renounceOwnership()
```

**Recommendation**

Inherit OpenZeppelin `Ownable2Step`. The two-step handover also protects against transferring to a mistyped or uncontrolled address. Deploy with a Safe multisig as owner from the outset, and consider a timelock on wallet-configuration changes so members can observe a pending treasury change before it takes effect.

**This cannot be retrofitted after deployment.** It is a few lines of inheritance now, or a full migration later.

**SEC-03 No pause and no rescue — the protocol has no incident response capability****HIGH**

|                   |                                                         |
|-------------------|---------------------------------------------------------|
| <b>Location</b>   | HNTRMembership.sol — contract-wide                      |
| <b>Category</b>   | Operational Security                                    |
| <b>Impact</b>     | High — converts recoverable incidents into total losses |
| <b>Likelihood</b> | Medium                                                  |

**Description**

There is no circuit breaker anywhere in the contract, and it is not upgradeable. If the signer key is compromised (SEC-01), if a protocol wallet is blacklisted (SEC-04), or if a logic flaw is found after launch, there is no way to stop `purchaseMembership` and `upgradeMembership` while a fix is arranged. Every incident runs to completion.

This finding matters mainly because of what it does to the others. SEC-01 is a serious but bounded problem if you can halt sales within minutes of detection; without a pause, the attacker collects 65% of revenue for as long as it takes to warn users off-chain and have them stop transacting voluntarily. The same applies to SEC-04. A pause switch does not prevent any of these incidents — it is what makes them survivable.

There is also no rescue function, so any ERC-20 sent directly to the contract — a routine user error given its role as a payment endpoint — is permanently unrecoverable, as is the rounding dust that accumulates on every sale.

**Recommendation**

Add `OpenZeppelin Pausable` with `whenNotPaused` on both membership entry points. **Deliberately leave** `withdrawCommissions` **unpausable**, so a pause can never trap member funds — this distinction is what separates a safety control from a rug vector.

Add an owner-only `rescueToken` that is explicitly barred from reducing the `usdt/usdc` balance below the sum of outstanding `withdrawableCommissions`. Track that sum in a running total so the invariant is cheap to enforce on-chain.

**SEC-04 Distribution assumes tokens received equals amount requested — insolvency risk****HIGH**

|                   |                                         |
|-------------------|-----------------------------------------|
| <b>Location</b>   | HNTRMembership.sol L218–269             |
| <b>Category</b>   | External Token Integration              |
| <b>Impact</b>     | High — members lose accrued commissions |
| <b>Likelihood</b> | Low — requires issuer action            |

**Description**

`_processPaymentAndDistribution` pulls amount via `safeTransferFrom`, then distributes percentages of that same amount without ever measuring what actually arrived.

Tether's contract contains a dormant `basisPointsRate` fee that the issuer can enable at any time by calling `setParams`. If it is ever switched on, every purchase distributes more than it received. The shortfall does not fail loudly — it is drawn silently from the pooled `withdrawableCommissions` of unrelated members, and the loss lands on whoever withdraws last. This is the one realistic path by which the commission ledger, which we otherwise verified as solvent (Section 5.3), becomes insolvent.

The related exposure is blacklisting. Both USDT and USDC can freeze arbitrary addresses. If `treasuryWallet`, `leadershipWallet`, `achievementWallet` or `poolWallet` is frozen, every purchase and upgrade reverts — a complete halt of protocol revenue, with no pause, no migration path, and no way to route around the frozen address other than an owner transaction that may itself arrive too late.

**Proof of Concept**

```
IERC20(token).safeTransferFrom(payer, address(this), amount);
// received may be < amount — never measured

IERC20(token).safeTransfer(treasuryWallet, (amount * 25) / 100);
IERC20(token).safeTransfer(leadershipWallet, (amount * 5) / 100);
IERC20(token).safeTransfer(achievementWallet, (amount * 5) / 100);
// ...all splits computed on the *requested* amount

// With a 0.1% fee enabled:
// received = 0.999 * amount
// paid out = 1.000 * amount worth of splits
// deficit -> silently drawn from other members' balances
```

**Recommendation**

Measure the delta. Snapshot `balanceOf(address(this))` before and after the pull and distribute the realised amount — a two-line change that makes the contract correct for any ERC-20 rather than for a specific pair.

For blacklisting, move the four protocol wallets to a pull-payment pattern, or wrap the transfers so a single frozen destination cannot block the sale. Pair with the pause switch from SEC-03 so the operator has time to reconfigure.

**SEC-05 No duplicate or self-referential upline rejection****MEDIUM**

|                   |                                   |
|-------------------|-----------------------------------|
| <b>Location</b>   | HNTRMembership.sol L235–263       |
| <b>Category</b>   | Business Logic / Defence in Depth |
| <b>Impact</b>     | Medium                            |
| <b>Likelihood</b> | Low — requires SEC-01             |

**Description**

The distribution loop never checks that upline addresses are distinct, nor that an upline differs from the buyer. A payload listing one address twelve times credits that address at every level — the full 65% of the sale. The buyer can also be their own upline: their tier is written to storage *before* distribution runs, so they qualify against their own gates.

On its own this is not exploitable, because only `companyWallet` can produce a valid payload. It is included because it is the missing invariant that would otherwise cap the damage from SEC-01. With a distinctness check in place, an attacker holding the signing key must spread stolen commissions across twelve separate addresses per sale and cannot simply funnel everything to one — a meaningful constraint on both the economics and the traceability of the attack. Defence in depth matters most precisely where the primary control is a single off-chain key.

**Proof of Concept**

```
for (uint256 i = 0; i < uplines.length && currentLevel <= 12; i++) {
    address upline = uplines[i];
    // no require(upline != payer)
    // no duplicate check against earlier entries
    if (qualified) {
        withdrawableCommissions[upline][token] += liquid;
        currentLevel++;
    }
}
```

**Recommendation**

Reject `upline == payer` outright. Enforce uniqueness across the list — with a twelve-element ceiling an  $O(n^2)$  inner comparison costs little, or use transient storage (EIP-1153) on a post-Cancun deployment.

## 7. Verified Secure — Attack Surface Cleared

The following attack classes were tested and found **not exploitable**. We list them because knowing what was checked and cleared is as useful as knowing what failed — and because several of these are the failure modes that account for most losses in comparable protocols.

| Attack Class                     | Result          | Basis                                                                                                                                                                                                                                                                                                                                             |
|----------------------------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Theft of accrued commissions     | Not possible    | No function transfers withdrawableCommissions to anyone but the balance holder. withdrawCommissions enforces msg.sender == user; the company-wallet sweep sends funds to the user, not the caller. Neither owner nor company wallet can drain the contract.                                                                                       |
| Reentrancy                       | Not exploitable | _withdraw zeroes the balance before transferring (CEI). Membership state is written before distribution. Both permitted tokens are non-callback ERC-20s fixed at construction as immutables. No guard is present, so this rests on the token allowlist rather than an explicit control — adding ReentrancyGuard would make the property explicit. |
| Signature replay                 | Blocked         | Payload binds block.chainid, address(this) and an operation tag, closing cross-chain, cross-contract and cross-function reuse. Same-message replay is blocked by state guards — see SEC-01 for why we consider this incidental rather than designed.                                                                                              |
| Signature malleability           | Blocked         | OpenZeppelin ECDSA v5 rejects high-s signatures and s-value manipulation.                                                                                                                                                                                                                                                                         |
| Hash collision in signed payload | Not possible    | abi.encode length-prefixes and offset-encodes both dynamic arrays, so no ambiguity exists between the uplines and ranks fields. abi.encodePacked would have been vulnerable here; it was correctly avoided.                                                                                                                                       |
| Free or underpriced membership   | Not possible    | Price is read from on-chain tierPrices, never supplied by the caller or the signature. On Ethereum mainnet both permitted tokens use 6 decimals, matching the price table.                                                                                                                                                                        |
| Front-running / signature theft  | Not possible    | Signatures bind the user address and both entry points require msg.sender == user, so an observed mempool signature cannot be used by a third party.                                                                                                                                                                                              |
| Commission over-distribution     | Not possible    | Verified symbolically in Section 5.3. Level percentages sum to exactly 65%; independent flooring guarantees outflow never exceeds inflow.                                                                                                                                                                                                         |
| USDT non-standard return value   | Handled         | SafeERC20 used on every transfer and transferFrom.                                                                                                                                                                                                                                                                                                |
| Integer overflow / underflow     | Not possible    | Solidity 0.8.x checked arithmetic throughout. The one subtraction of concern, tierPrices[newTier] - tierPrices[u.tier], is guarded by require(newTier > u.tier).                                                                                                                                                                                  |

## 8. Trust Model

HNTRMembership is a permissioned protocol. Members trust two off-chain keys, and in the current design neither trust relationship can be reduced by on-chain means. We set this out plainly because it is the thing a prospective member most needs to understand.

| Role                  | Powers                                                                                                     | Bounded By                                                                                               |
|-----------------------|------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| owner (EOA)           | Set treasury, leadership, achievement, pool and company wallets at any time, with immediate effect.        | Cannot touch accrued member balances. Cannot mint memberships. Role is permanent and unmovable (SEC-02). |
| companyWallet         | Sole valid signer for every purchase and upgrade. Fully determines the upline list and each upline's rank. | Cannot touch existing balances. Controls the destination of all future commissions (SEC-01).             |
| companyWallet (sweep) | Force-withdraw any member's commissions.                                                                   | Funds are sent to the member, never the caller. Not a theft vector.                                      |
| poolWallet            | Receives 20% of every commission; settlement to members is off-chain.                                      | By design, per client confirmation. Carries no on-chain enforcement.                                     |

### What members cannot lose

The positive case deserves the same clarity as the negative one. **Accrued liquid commissions are safe from operator action.** There is no privileged function that moves `withdrawableCommissions` to anyone but the balance holder, and `withdrawCommissions` requires `msg.sender == user`. The centralisation risk in this protocol is over *future* payment flows and over the off-chain-settled pool — not over funds already credited on-chain.

### Recommendations

- Move owner to a Safe multisig — which first requires implementing ownership transfer (SEC-02).
- Place wallet-configuration changes behind a 48-hour timelock so members can observe pending changes.
- Support multiple signers with individual revocation; hold keys in an HSM or KMS (SEC-01).
- Publish the trust model in user-facing documentation, including that the locked 20% carries no on-chain guarantee.

## 9. Remediation Roadmap

| Priority                   | Findings       | Action                                                                                                                                                                                                           | Effort   |
|----------------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| P0 — Cannot be added later | SEC-02, SEC-03 | Inherit Ownable2Step and Pausable; add a bounded rescueToken. Leave withdrawCommissions unpausable. Deploy with a multisig as owner.                                                                             | 2–3 days |
| P1 — Pre-launch            | SEC-01, SEC-04 | Add per-user nonce and a global invalidation epoch to the signed struct; migrate to EIP-712; shorten deadlines; support multiple revocable signers. Switch distribution to measured balance deltas.              | 3–4 days |
| P2 — Pre-launch            | SEC-05         | Reject self and duplicate uplines to cap the blast radius of a signer compromise.                                                                                                                                | 0.5 day  |
| P3 — Verification          | All            | Add tests for each finding above, plus invariant fuzzing on the solvency property from Section 5.3 (contract balance $\geq$ sum of withdrawableCommissions after every operation). Re-audit the remediated code. | 4–6 days |

*Estimates assume one experienced Solidity engineer and exclude the re-audit. Appendix A items can be folded into the same work at negligible marginal cost but are not launch blockers.*



## 10. Conclusion

HNTRMembership is a well-built contract. The commission arithmetic is correct and provably solvent, SafeERC20 is applied consistently, the signature payload is bound properly against replay, and accrued member balances cannot be touched by any privileged role. We found no critical vulnerability and no path for an unprivileged attacker to steal funds or obtain a membership for free. Section 7 lists ten attack classes we tested and cleared.

Our concern is not that the contract is likely to be attacked. It is that **it was built without any way to respond if it is**. SEC-01, SEC-02 and SEC-03 are three views of one gap: a single off-chain key authorises all commission flow, that key cannot be rotated if the owner key is lost, the owner role itself can never move to a multisig, and nothing can be paused while any of this is sorted out. SEC-04 supplies the most plausible trigger — a dormant USDT transfer fee that would quietly make the commission ledger insolvent.

The asymmetry here is what makes the recommendation straightforward. Ownable2Step and Pausable are two inherited contracts and a handful of modifiers — hours of work, no architectural change, no effect on the commission logic. Deployed without them, the same fixes are impossible at any price, because the contract is not upgradeable and the owner role cannot be moved. That is the entire argument for treating them as launch blockers.

SEC-01 and SEC-04 require slightly more work — a nonce and an invalidation epoch in the signed struct, and a switch to measured balance deltas — but both are well-trodden patterns with reference implementations. SEC-05 is half a day and meaningfully limits what a stolen signing key can do.

**Recommendation:** complete P0 and P1 before mainnet, add the tests in Section 9, and commission a re-audit of the remediated code. With those changes we would expect a residual risk rating of **Low**, with the remaining exposure being the operator centralisation inherent to the design — which should be disclosed to members rather than engineered away.

**DEPLOY-BLOCKING: SEC-02, SEC-03 — neither can be retrofitted post-deployment**

## 11. Disclaimer

---

This report is provided by Stellar Code on an “as is” basis and reflects the state of the reviewed source code at the time of assessment. A security audit is a best-effort, time-boxed activity and cannot prove the absence of vulnerabilities. It is not a warranty, guarantee, endorsement, or investment advice.

The assessment covered only the files listed in Section 3. The off-chain signing backend, the front-end, key-management practices, deployment infrastructure and the economic sustainability of the commission model were outside scope.

Third-party dependencies — OpenZeppelin Contracts v5 and the USDT and USDC token contracts — were assumed to behave as documented, other than the known non-standard behaviours discussed in SEC-04.

Findings are scoped to deployment on Ethereum mainnet against 6-decimal USDT and USDC. Deployment to any other chain, or configuration with any other token, invalidates parts of this analysis — see Appendix A, item 7. Any modification to the reviewed code invalidates the findings in this report.

**Non-legal observation.** The protocol implements a multi-level commission structure in which participants pay a membership fee and earn from recruitment across twelve levels. Such structures receive specific and varying regulatory treatment across jurisdictions. Stellar Code offers no legal opinion and did not assess this; independent counsel should be engaged in every jurisdiction where the protocol will be marketed.

*This document is confidential and intended solely for the named client. Redistribution requires written consent from Stellar Code.*

## Appendix A — Non-Security Observations

The following were identified during review but have **no path to loss of funds or loss of protocol control**, and are therefore excluded from the findings above. They are recorded for completeness and can be addressed alongside the P0–P2 work at negligible marginal cost. Items marked † were rated as findings in report v1.0 and reclassified in v2.0.

| #   | Observation                                                                                                                                                                                                                            | Type             |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| 1†  | setWallets / setCompanyWallet accept address(0). The deploy script configures and asserts these, and an owner who can redirect funds does not need address(0) to do harm — so this is a footgun, not a vulnerability.                  | Validation       |
| 2†  | withdrawCompanyWallet's 30-day guard short-circuits on lastClaimedAt == 0, so a never-claimed member is immediately sweepable. Funds go to the member, so there is no theft — but the stated protection does not work for new earners. | Logic            |
| 3†  | getOverdueWallets scans an unbounded allUsers array twice; becomes uncallable past roughly 3–4k members. It is a view function and the same data is derivable from events.                                                             | Scalability      |
| 4†  | Tier prices are immutable — no setter exists, so repricing requires redeployment.                                                                                                                                                      | Flexibility      |
| 5†  | uplines array length is uncapped; a long unqualified list makes the buyer's own transaction expensive.                                                                                                                                 | Gas              |
| 6†  | lockedCommissions only increments, so it records lifetime accrual rather than outstanding share, and duplicates data already emitted in CommissionEarned at ~3k–22k gas per upline.                                                    | Accounting       |
| 7†  | The 6-decimal price assumption is correct for Ethereum mainnet but unenforced in code. A one-line constructor assertion on decimals() would prevent a silent mispricing if the same bytecode is later deployed to an 18-decimal chain. | Portability      |
| 8†  | No ReentrancyGuard. Not exploitable with the current token allowlist (see Section 7), but the safety property is incidental rather than explicit.                                                                                      | Defence in depth |
| 9†  | Rounding dust accumulates permanently; unrecoverable absent the rescue function recommended in SEC-03.                                                                                                                                 | Accounting       |
| 10† | withdrawCompanyWallet emits both CommissionWithdrawn and CompanyWalletWithdrawn for one movement, double-counting for naive indexers.                                                                                                  | Events           |
| 11† | require(price > 0, "Tier price not set") is unreachable — every tier is priced in the constructor and no setter exists.                                                                                                                | Dead code        |
| 12† | The user parameter on both entry points is redundant given require(msg.sender == user); it wastes calldata and invites integrators to assume delegated calls are possible.                                                             | API design       |
| 13† | Events are under-indexed — token, tier and level are natural filter dimensions; wallet-configuration events index nothing.                                                                                                             | Events           |
| 14† | Configuration events emit only new values, not before/after, complicating treasury-change monitoring.                                                                                                                                  | Events           |
| 15† | Floating pragma ^0.8.24; pin to the exact compiler used for deployment for reproducible verification.                                                                                                                                  | Build            |
| 16† | IHNTRMembership omits the administrative surface (setWallets, setCompanyWallet, getOverdueWallets) and all state getters.                                                                                                              | Documentation    |
| 17† | onlyCompanyWallet on the view function getOverdueWallets provides no confidentiality — all inputs are public storage and eth_call can spoof from.                                                                                      | Design           |
| 18† | String revert reasons rather than custom errors; converting reduces deployment size and revert-path gas.                                                                                                                               | Gas              |
| 19† | test/Mocks.sol contains unused MockSeaport and MockSwapRouter from an unrelated codebase. MockERC20 inherits OZ's 18-decimal default while the contract assumes 6, so the suite tests a configuration that does not match mainnet.     | Test quality     |

| #   | Observation                                                                                                                          | Type          |
|-----|--------------------------------------------------------------------------------------------------------------------------------------|---------------|
| 20† | Sparse NatSpec. The compression model, tier/rank gates and 80/20 split are non-obvious and documented only in brief inline comments. | Documentation |

*Item 19 is worth singling out: because MockERC20 uses 18 decimals, the existing test suite validates the protocol against a token configuration that will never be deployed. Pinning the mock to 6 decimals is a small change that makes every existing test more meaningful.*

**Stellar Code — Security Division**

Report v2.0 · Issued 18 July 2026 · Confidential